TOPIC 05.0: BINARY SEARCH FOUNDATION & EVOLUTION

Foundation & Templates

>

🧰 REUSABLE BINARY SEARCH UTILITIES (SIDE-BY-SIDE TEMPLATES)

Lower Bound (First Index where arr[i] >= target):

```
// Lower Bound: First index where arr[i] ⩾ target
int lowerBound(int[] arr, int target) {
    int lo = 0, hi = arr.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2; // Safe Mid
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

Upper Bound (First Index where arr[i] > target):

```
// Upper Bound: First index where arr[i] > target
int upperBound(int[] arr, int target) {
    int lo = 0, hi = arr.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2; // Safe Mid
        if (arr[mid] ⩽ target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

💡 AHA MOMENT #1: SEARCH SPACE REDUCTION & BOUNDARY TRANSITION

Binary Search is NOT just an array search! It is a general **search-space reduction paradigm**. We evaluate the middle element `mid` against a monotonic property (e.g. `F F F F T T T T`) to discard half (N/2) of the search space in O(1) time, shrinking N elements to 1 in exactly $log₂(N)$ steps!

📌 PATTERN EVOLUTION: HOW ALGORITHMS EVOLVE

`Linear Scan O(N) ➔ Binary Search O(log N) ➔ Binary Search on Answer O(N log M)`

- **Linear Scan:** Inspects elements sequentially from 0 to N-1 (O(N) time).
- **Binary Search:** Halves search space interval at each step on monotonic inputs (O(\log N) time).
- **Binary Search on Answer:** Binary searches integer solution space $[L, R]$ using feasibility checks in O(N \log M)!

🎨 MONOTONIC TRANSITION BOUNDARY (F ➔ T)

F	F	F	F	T ★	T	T

↑ Binary search pinpoints the FIRST TRUE transition boundary index in O(\log N)!

1. Overflow-Safe Mid Formula: `int mid = left + (right - left) / 2;` avoids integer overflow.

2. Closed Interval Rule: With `left = 0, right = N - 1`, loop condition MUST be `while (left ⩽ right)`.

📌 **KEY TAKEAWAY:** For $N = 1,000,000$, Linear Scan takes 1,000,000 steps whereas Binary Search takes only ~20 steps!

>

1 CLASSIC BINARY SEARCH TEMPLATE

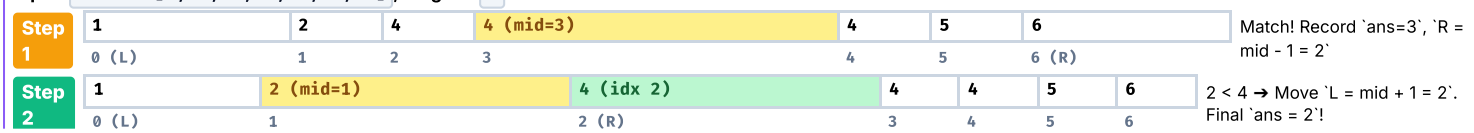
```
public int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left ⩽ right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

2 FIRST OCCURRENCE (LOWER BOUND)

```
public int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1, ans = -1;
    while (left ⩽ right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid; right = mid - 1; // Record & search left!
        } else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return ans;
}
```

🎨 VISUAL TRACE: FIRST OCCURRENCE OF TARGET = 4

Input: `nums = [1, 2, 4, 4, 4, 5, 6]`, **Target =** `4`



TOPIC 05.0: BINARY SEARCH FOUNDATION & EVOLUTION

Core Patterns — Part 1 & Part 2

>

💡 **AHA MOMENT #4: SEARCHING SOLUTION RANGE $[L, R]$ & 2D MATRIX FLATTENING**

- **BS on Answer:** Binary search integer solution range $[L, R]$ using boolean `isValid(mid)` helper.
- **1D Matrix Mapping:** Convert 1D `mid` to 2D coordinates in O(1): `row = mid / cols` and `col = mid % cols`!

1 BS ON ANSWER JAVA TEMPLATE

```
public int bsOnAnswerTemplate(int[] nums, int limit) {
    int left = 1, right = getMax(nums), ans = right;
    while (left ⩽ right) {
        int mid = left + (right - left) / 2;
        if (isValid(nums, mid, limit)) {
            ans = mid; right = mid - 1; // Try smaller!
        } else left = mid + 1;
    }
    return ans;
}
```

2 1D MATRIX FLATTENING (LC 74)

```
public boolean searchMatrix(int[][] matrix, int target) {
    int rows = matrix.length, cols = matrix[0].length;
    int left = 0, right = rows * cols - 1;
    while (left ⩽ right) {
        int mid = left + (right - left) / 2;
        int val = matrix[mid / cols][mid % cols];
        if (val == target) return true;
        else if (val < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

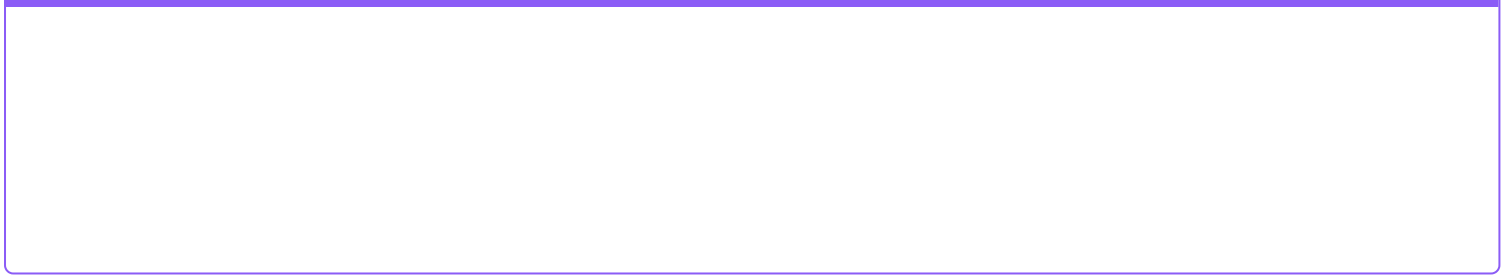
📊 MATRIX SEARCH COMPLEXITY COMPARISON

Matrix Structure	Algorithm Technique	Time Complexity	Space Complexity
Fully Sorted (Row i end < Row i+1 start)	1D Flattening Binary Search	`O(log(R * C))`	`O(1)`
Rows & Cols Independently Sorted	Top-Right Staircase Walk	`O(R + C)`	`O(1)`

📌 **KEY TAKEAWAY:** BS on Answer converts hard optimization problems into feasibility checks in O(N \log M) time!

>

🌳 BINARY SEARCH INTERVIEW DECISION TREE



1 BUILT-IN JAVA APIS

- `Arrays.binarySearch(arr, key)` returns index >= 0 if found, or `-(insertion_point) - 1` if missing!
- `Collections.binarySearch(list, key, comparator)` supports custom Object comparators.

2 TOP 5 BINARY SEARCH BUGS

1. Overflow in `(left + right)/2`.
2. Infinite stuck loop via `left = mid`.
3. Search on unsorted inputs.
4. Loop bound mismatches.
5. Missing empty input checks.

🗣️ **FAANG COACH TIP:** `Arrays.binarySearch()` returning `-insertionPoint - 1` lets you find insert index in 1 line!

TOPIC 05.0: BINARY SEARCH FOUNDATION & EVOLUTION

Core Patterns — Part 3 & Part 4

>

1. LEETCODE 704 — BINARY SEARCH

Easy ★★★★★ Classic BS Amazon Meta

Problem:

Given a sorted array

`nums`

and

`target`

, return index or -1.

Example:

`nums = [-1,0,3,5,9,12], target = 9`

→ Output:

`4`

.

```
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left ⩽ right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

2. LEETCODE 35 — SEARCH INSERT POSITION

Easy ★★★★★ Lower Bound Google Apple

Problem:

Return index if target found. If not, return index where it would be inserted in order.

Example:

`nums = [1,3,5,6], target = 5`

→

`2`

.

`target = 2`

→

`1`

.

```
public int searchInsert(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left ⩽ right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return left; // left holds insertion index!
}
```

3. LEETCODE 374 — GUESS NUMBER HIGHER OR LOWER

Easy ★★★★★ API Evaluation Google

Problem:

Guess number between 1 and n using pre-defined

`guess(int num)`

API.

```
public int guessNumber(int n) {
    int left = 1, right = n;
    while (left ⩽ right) {
        int mid = left + (right - left) / 2;
        int res = guess(mid);
        if (res == 0) return mid;
        else if (res < 0) right = mid - 1;
        else left = mid + 1;
    }
    return -1;
}
```

💡 **AHA MOMENT (LC 35):** On loop finish, `left` pointer ALWAYS points to the exact insertion index!

>

4. LEETCODE 74 — SEARCH A 2D MATRIX

Medium ★★★★★ 1D Flattening Meta Amazon

```
public boolean searchMatrix(int[][] matrix, int target) {
    int rows = matrix.length, cols = matrix[0].length;
```

>

6. LEETCODE 153 — FIND MINIMUM IN ROTATED SORTED ARRAY

Medium ★★★★★ **Pivot Search** Meta Amazon

Problem:

Given a rotated sorted array

`nums`

of unique elements, return minimum element.

Example:

`nums = [3,4,5,1,2]`

→

`1`

.

```
public int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[right]) left = mid + 1;
        else right = mid;
    }
    return nums[left];
}
```

7. LEETCODE 875 — KOKO EATING BANANAS

Medium ★★★★★ **BS on Answer** Google Meta

Problem:

Min eating speed

`k`

to eat all banana piles within

`h`

hours.

Example:

`piles = [3,6,7,11], h = 8`

→

`4`

.

```
public int minEatingSpeed(int[] piles, int h) {
    int left = 1, right = 0;
    for (int p : piles) right = Math.max(right, p);
    int res = right;
    while (left ⩽ right) {
        int k = left + (right - left) / 2;
        long hours = 0;
        for (int p : piles) hours += (p + k - 1) / k;
        if (hours ⩽ h) {
            res = k; right = k - 1; // Try slower
        } else left = k + 1;
    }
    return res;
}
```

💡 **AHA MOMENT (LC 875):** Integer ceiling division formula: `(p + k - 1) / k` avoids floating point calculations!

>

Medium ★★★★☆ **Slope Search** Meta Amazon

Problem:

Find index of any peak element strictly greater than its neighbors.

Example:

```
nums = [1,2,3,1]
```

→

```
2
```

.

```
public int findPeakElement(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] < nums[mid + 1]) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

9. LEETCODE 981 — TIME BASED KEY-VALUE STORE

Medium ★★★★★ **List BS Search** Google Meta

Problem:

Store key-value pairs with timestamps and retrieve value with

```
timestamp_prev ≤ timestamp
```

.

```
class TimeMap {
    private Map<String, List<Pair>> map = new HashMap<>();
    public void set(String key, String value, int timestamp) {
        map.computeIfAbsent(key, k → new ArrayList<>()).add(new
Pair(timestamp, value));
    }
    public String get(String key, int timestamp) {
        if (!map.containsKey(key)) return "";
        List<Pair> list = map.get(key);
        int left = 0, right = list.size() - 1; String res = "";
        while (left ≤ right) {
            int mid = left + (right - left) / 2;
            if (list.get(mid).time ≤ timestamp) {
                res = list.get(mid).val; left = mid + 1;
            } else right = mid - 1;
        }
        return res;
    }
    private static class Pair { int time; String val; Pair(int t,
String v){time=t;val=v;}}
}
```

📌 **KEY TAKEAWAY:** Peak element search moves towards ascending slope (`nums[mid] < nums[mid+1]`) because a peak MUST exist in that direction!

>

10. LEETCODE 410 — SPLIT ARRAY LARGEST SUM

Hard ★★★★★ **BS on Answer** Google Amazon

Problem:

Split array into

```
k
```

non-empty contiguous subarrays minimizing largest sum.

```
public int splitArray(int[] nums, int k) {
    int left = 0, right = 0;
    for (int n : nums) { left = Math.max(left, n); right += n; }
    int res = right;
    while (left ≤ right) {
        int mid = left + (right - left) / 2;
        if (canSplit(nums, k, mid)) { res = mid; right = mid - 1;
}
        else left = mid + 1;
    }
    return res;
}
private boolean canSplit(int[] nums, int k, int maxSum) {
    int count = 1, currentSum = 0;
    for (int n : nums) {
        if (currentSum + n > maxSum) { count++; currentSum = n; }
        else currentSum += n;
    }
    return count ≤ k;
```

TOPIC 05.0: BINARY SEARCH FOUNDATION & EVOLUTION

Dry Run, Proofs & Cheat Sheet

>

⭐ THE 4 GOLDEN LAWS OF BINARY SEARCH

1. **Overflow-Safe Mid:** Always calculate `int mid = left + (right - left) / 2;`.
2. **Inclusive Bounds Rule:** With `left = 0, right = N - 1`, loop MUST be `while (left <= right)` and update `left = mid + 1`, `right = mid - 1`.
3. **Boundary Record Trick:** For first/last occurrence, record `ans = mid` when matched and continue searching leftward/rightward.
4. **Binary Search on Answer:** Search integer solution range $[L, R]$ with boolean `isValid(mid)` helper!

⛔ WHEN BINARY SEARCH FAILS (DO NOT USE!)

- **Unsorted Input Array:** Input is unsorted and index order must be preserved ➔ Use **HashMap**!
- **Non-Monotonic Predicate:** `isValid(x)` fluctuates unpredictably ➔ Use **Dynamic Programming**!

📊 NEETCODE & BLIND 75 CONFIDENCE TRACKER

Easy LC 704 Binary Search	Easy LC 35 Search Insert	Easy LC 374 Guess Number
Med LC 74 Search 2D Matrix	Med LC 33 Search Rotated	Med LC 153 Find Min Rotated
Med LC 875 Koko Bananas	Med LC 162 Find Peak	Med LC 981 Time Key Value
Hard LC 410 Split Array Max	Hard LC 4 Median Two Sorted	

👉 Next Master Topic: Topic 06 → Stack & Queues Masterclass

🎉 **TOPIC 05 BINARY SEARCH COMPLETE!** Turn the page to **Topic 06: Stack & Queues Masterclass**!